机器学习 1：机器学习的一般步骤（手写数字数据集案例）

一、 scikit_learn 的介绍

本实验在 Anaconda3 环境下进行，Anaconda3 已包含 scikit_learn 包，无需另外下载安装。

scikit-learn 是一个开源的 Python 语言机器学习工具包。它涵盖了几乎所有主流机器学习算法的实现，并且提供了一致的调用接口。

它基于 Numpy 和 SciPy 等 Python 数值计算库，提供了高效的算法实现。

总结起来，scikit-learn工具包有以下几个优点：

(1) 文档齐全：官方文档齐全，更新及时。

(2) 接口易用：针对所有的算法提供了一致的接口调用规则，不管是 KNN、K-Means 还是 PCA。

(3) 算法全面：涵盖主流机器学习任务的算法，包括回归算法、分类算法、聚类分析、数据降维处理等。

当然，scikit-learn 不支持分布式计算，不适合用来处理超大型数据。

二、 机器学习的一般步骤

1. 加载数据集

实验使用 digits 数据集，digits 数据集是一个手写数字的数据集。

代码：

```
from sklearn.datasets import load_digits
from matplotlib import pyplot as plt
digits=load_digits()
images_and_labels=list(zip(digits.images,digits.target))
plt.figure(figsize=(8,6),dpi=200)
for index,(image,label)in enumerate(images_and_labels[:8]):
    plt.subplot(2,4,index+1)
    plt.axis('off') # 关闭坐标轴
    plt.imshow(image,cmap=plt.cm.gray_r,interpolation='nearest')
    plt.title('Digit:%i'%label,fontsize=20)
```

注释：

> # digits.images: 包含了所有的图像数据，每个图像是一个 8x8 的二维数组。
>
> # digits.target: 包含了每个图像对应的真实标签（即这个数字到底写的是几）。
>
> # zip() 函数将图像和对应的标签一一配对，然后 list() 将这些配对的数据转换为列表。
>
> # 这样 images_and_labels 里的每一个元素都是一个包含 (图像矩阵, 标签) 的元组。
>
> # enumerate() 函数会同时返回当前循环的索引号（index，从 0 开始）以及对应的元素（这里是 image 和 label）。
>
> # cmap=plt.cm.gray_r: 设置颜色映射（Color Map）为反转的灰度图（gray_r 中的 r 代表 reverse）。这样原本数值大的地方显示为黑色，数值小的地方显示为白色，符合我们看白纸黑字的习惯。
>
> # interpolation='nearest': 设置图像插值方法为“最近邻插值”。因为原图只有 8x8 像素，拉伸显示时使用这种方法可以保持像素的方块感，不会显得模糊。

digits.data.shape：

> scikit-learn 使用Numpy 的 array 对象来表示数据，所有的图片数据保存在里，每个元素都是一个 8x8 尺寸的图片。我们在进行机器学习时，需要把数据保存为格式的 array 对象，针对手写数字识别这个案例，scikit-learn 已经为我们转换好了，把特征数据保存在 digits.data 数据里，可以通过以下两行代码来查看它的数据格式。

代码：

```
print("shape of raw image_data:{0}".format(digits.images.shape))
print("shape if data:{0}".format(digits.data.shape))
```

输出结果：

```
shape of raw image_data:(1797, 8, 8)
shape if data:(1797, 64)
```

2. 划分训练集和测试集

在开始训练模型前，需要对数据集进行划分，将数据集划分为训练集和测试集。可以使用以下两行代码对数据集进行划分，划分出 20%作为测试数据集，80%作为训练数据集。

代码：

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(digits.data,digits.target,test_size=0.20,random_state=2)
```

3. 训练模型

划分完训练集和测试集后，我们就可以使用 svc 学习机器学习模型，使用准确率对模型进行测试。

代码：

```
from sklearn.svm import SVC
#将逻辑回归改为 SVC 分类
clf=SVC(kernel='rbf',gamma='scale',random_state=42)
clf.fit(X_train,y_train)
```

4. **测试模型**

代码：

```
accuracy=clf.score(X_test,y_test)
print('Accuracy score of the {} is {:.2f}'.format(clf.__class__.__name__,accuracy))
```

输出结果：

```
Accuracy score of the SVC is 0.98
```

可以发现，svc 比原例的逻辑回归准确率更高（逻辑回归：0.94），说明 svc 模型比逻辑回归模型在该问题上适配度更好，分类准确性更高。

除此之外，还可以直接把测试数据集里的部分图片显示出来，并且在图片的左下角显示预测值，右下角显示真实值。

代码：

```
y_pred=clf.predict(X_test)
fig,axes=plt.subplots(4,4,figsize=(8,8))
fig.subplots_adjust(hspace=0.1,wspace=0.1)
for i,ax in enumerate(axes.flat):
    ax.imshow(X_test[i].reshape(8,8),cmap=plt.cm.gray_r,interpolation='nea
rest')
    ax.text(0.05,0.05,str(y_pred[i]),fontsize=32,transform=ax.transAxes,
        color='green' if y_pred[i] == y_test[i] else 'red')
    ax.text(0.8,0.05,str(y_test[i]),fontsize=32,transform=ax.transAxes,color=
'black')
    ax.set_xticks([])
    ax.set_yticks([])

plt.show()
```

5. 模型保存和加载

当我们对模型的准确度感到满意后，就可以把模型保存下来。这样下次需要预测时，可以直接加载模型来进行预测，而不是重新训练 一遍模型。

代码：

```
import joblib
joblib.dump(clf,'digits_svm.pkl')

#演示调用
clf2=joblib.load('digits_svm.pkl')
accuracy2 = clf2.score(X_test,y_test)
print('test: Accuracy score of the {} is {:.2f}'.format(clf2.__class__.__name__,accuracy2))
```

输出结果：

```
test: Accuracy score of the SVC is 0.98
```

6. 模型更改

scikit-learn 的模型 API 在应用中是相似的。因此，我们通过随机森林分类器 RandomForestClassifier 轻松替换逻辑回归 LogisticRegression 分类器。这些更改很小，仅与分类器实例的创建有关。

代码：

```
from sklearn.ensemble import RandomForestClassifier # RandomForestClassifier 轻松替换 LogisticRegression 分类器
clf3 = RandomForestClassifier(n_estimators=100, n_jobs=-1, random_state=42)
clf3.fit(X_train, y_train)
accuracy3 = clf3.score(X_test, y_test)

print('Accuracy score of the {} is {:.2f}'.format(clf3.__class__.__name__, accuracy3))
```

输出结果：

```
Accuracy score of the RandomForestClassifier is 0.96
```